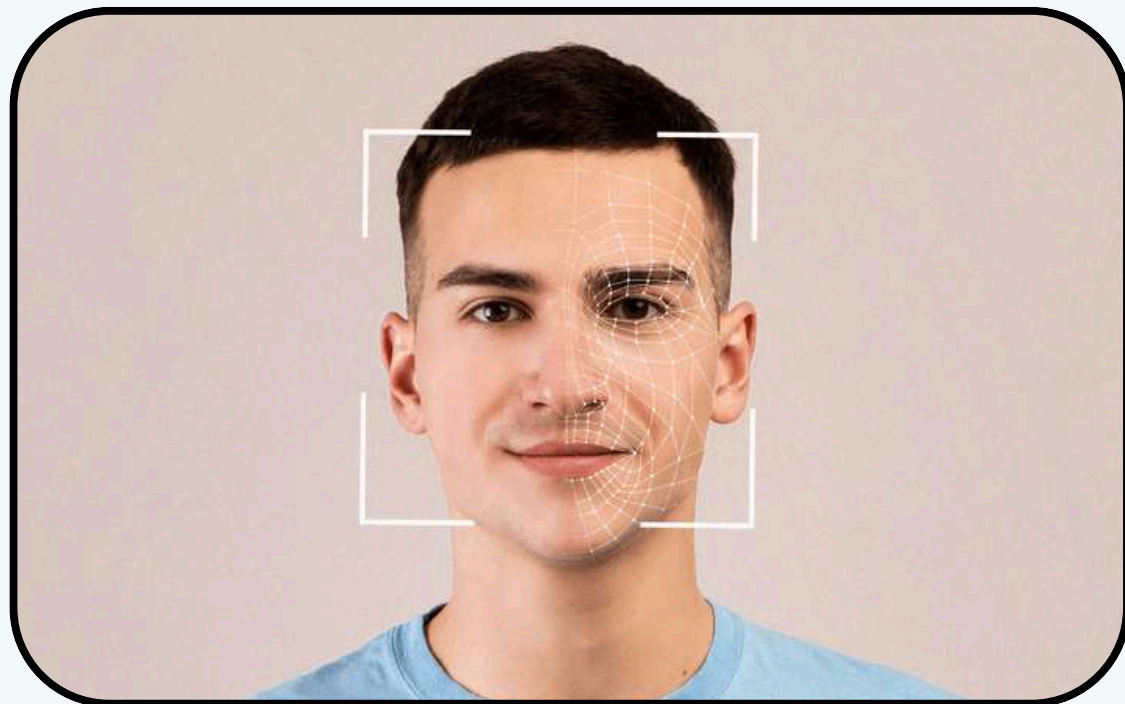


EDGE DETECTION DAN SHARPENING

Menggunakan Citra Wajah (Face Detection)

Pengolahan Citra Medika (A)



**Jeremia Christ
Immanuel Manalu**

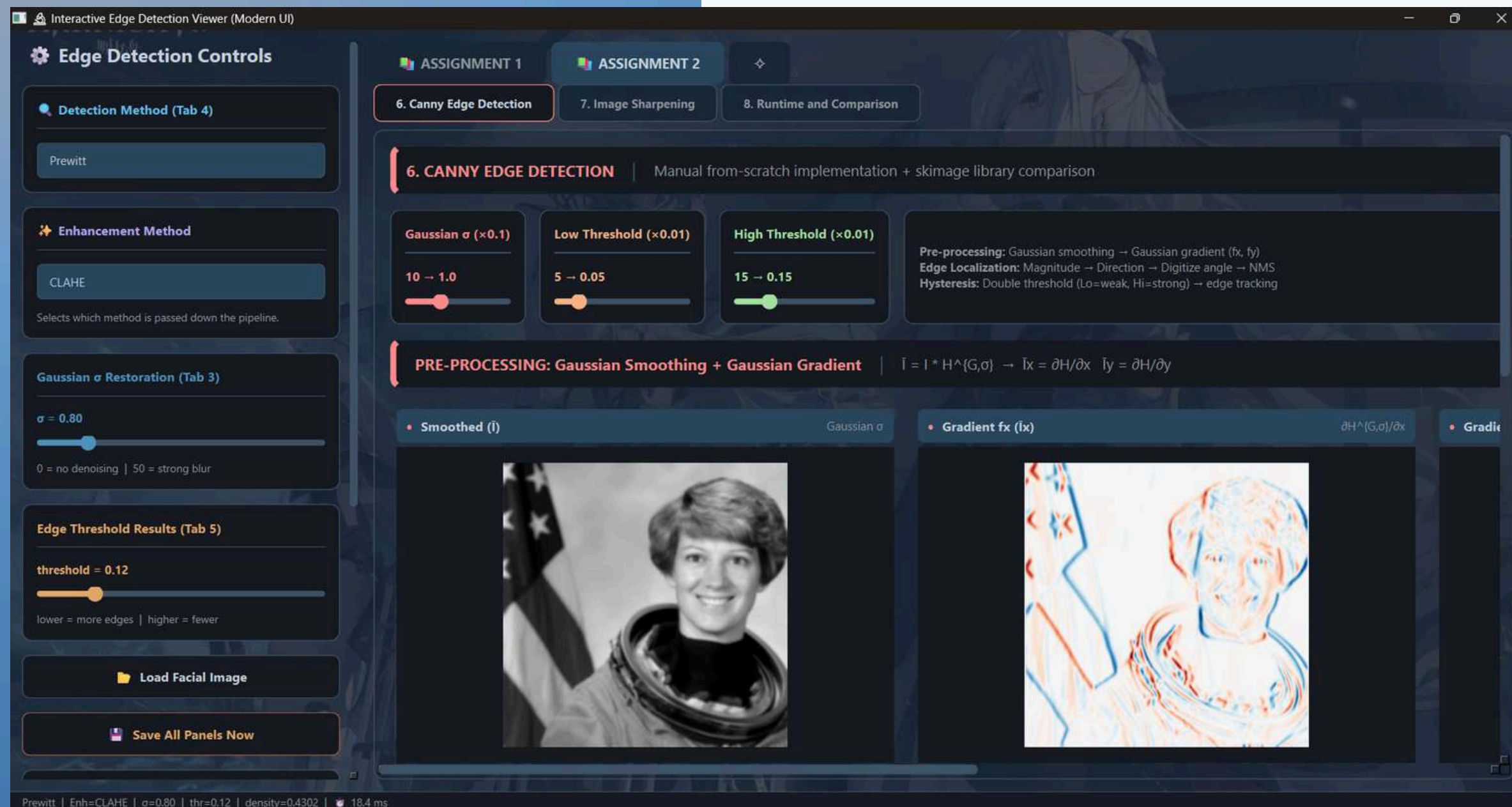


5023231017

01

PROGRAM

User Interface



```
root/  
├── main.py  
├── config.py  
├── core/  
│   ├── __init__.py  
│   ├── math_ops.py  
│   ├── enhancement.py  
│   ├── edge_detection.py  
│   ├── sharpening.py  
│   └── pipeline.py  
├── gui/  
│   ├── __init__.py  
│   ├── components.py  
│   └── main_window.py  
├── requirements.txt  
└── README.md
```

Struktur Direktori

*Lanjutan dari Assignment 1

CANNY EDGE DETECTION

01 Pre-processing (Gaussian Smoothing)

Menghilangkan noise sebelum deteksi tepi.

$$\bar{I} = I * H^{G,\sigma}$$

```
# Step 1: Gaussian smoothing  $\bar{I} = I * H^{G,\sigma}$ 
smoothed = manual_gaussian_filter(gray, sigma)
```

```
# Step 2: Gaussian gradient kernels (analytic derivatives)
#  $\bar{I}_x = -(x/\sigma^2) \cdot \exp(-(x^2+y^2)/\sigma^2)$ 
#  $\bar{I}_y = -(y/\sigma^2) \cdot \exp(-(x^2+y^2)/\sigma^2)$ 
k = max(1, int(np.ceil(3.0 * sigma)))
ax = np.arange(-k, k + 1, dtype=np.float64)
xx, yy = np.meshgrid(ax, ax)
gauss = np.exp(-(xx**2 + yy**2) / (2.0 * sigma**2))
gx_kernel = -(xx / sigma**2) * gauss
gy_kernel = -(yy / sigma**2) * gauss

fx = manual_convolve2d(smoothed, gx_kernel)
fy = manual_convolve2d(smoothed, gy_kernel)
```

Gaussian Gradient (f_x, f_y) 02

Menghitung turunan analitik menggunakan matriks mesh grid.

$$f_x = \frac{\partial H}{\partial x}, f_y = \frac{\partial H}{\partial y}$$

CANNY EDGE DETECTION

03 Magnitude dan Direction

Menghitung kuat tepi (Emag) dan arah sudut (Φ).

$$\Phi(u, v) = \arctan 2(I_y, I_x)$$

Arah disederhanakan ke 4 kuadran (Horizontal, Vertikal, Diagonal↗, Diagonal↘). Hanya mempertahankan nilai piksel maksimum lokal di sepanjang arah gradien.

```
# Step 3: Gradient magnitude Emag = √(fx² + fy²)
mag = np.hypot(fx, fy)
mag_max = mag.max() + 1e-9
mag_norm = mag / mag_max

# Step 4: Gradient direction Φ(u,v) = arctan2(Iy, Ix) + 180
angle_deg = np.rad2deg(np.arctan2(fy, fx)) + 180.0 # → [0°, 360°)
```

Angle Quantization dan NMS 04

```
# Step 5: Digitize angle → 4 directions
quantized = _digitize_angle(angle_deg)
```

```
# Step 6: Non-Maximum Suppression
nms = _non_max_suppression(quantized, mag)
nms_norm = nms / mag_max
```

05 Double Thresholding dan Hysteresis

Menggunakan ambang Low dan High. Tepi lemah (Weak) hanya dipertahankan jika terhubung (terkoneksi) dengan tepi kuat (Strong).

```
# Step 7: Double Thresholding
t_hi_abs = t_hi * mag_max
t_lo_abs = t_lo * mag_max
double_thresh = _double_threshold(nms, t_lo_abs, t_hi_abs)
```

```
# Step 8: Hysteresis Edge Tracking
final_edges = _hysteresis(double_thresh)
```


IMAGE SHARPENING (LAPLACIAN)

Penajaman menggunakan turunan orde kedua (Laplacian) untuk menyorot transisi intensitas yang cepat.

01 Formula Umum

$$I' = I - w \cdot (H^L * I)$$

di mana w adalah weight/bobot.

```
lap_full = manual_convolve2d(blur, HL)
sharp_full = np.clip(gray - weight * lap_full, 0.0, 1.0)
```

```
# Separable Laplacian kernels (1-D)
HL_x = np.array([[1.0, -2.0, 1.0]])
HL_y = np.array([[1.0], [-2.0], [1.0]])

lap_x = manual_convolve2d(blur, HL_x)
lap_y = manual_convolve2d(blur, HL_y)
lap_xy_sep = lap_x + lap_y
sharp_sep = np.clip(gray - weight * lap_xy_sep, 0.0, 1.0)
```

02 Varian Kernel (H^L)

- H4: Konektivitas 4-arah (Center -4).
- H8: Konektivitas 8-arah (Center -8).
- H12: Berbobot (Weighted 8-connectivity).

```
# Full 2-D Laplacian kernel
if kernel_type == "H8":
    HL = np.array([[1., 1., 1.],
                   [1., -8., 1.],
                   [1., 1., 1.]])
elif kernel_type == "H12":
    HL = np.array([[1., 2., 1.],
                   [2., -12., 2.],
                   [1., 2., 1.]])
else: # H4 (default, matches lecture HL = Hx + Hy)
    HL = np.array([[0., 1., 0.],
                   [1., -4., 1.],
                   [0., 1., 0.]])

lap_full = manual_convolve2d(blur, HL)
sharp_full = np.clip(gray - weight * lap_full, 0.0, 1.0)
```

UNSHARP MASKING (USM)

Teknik klasik fotografi yakni mengurangkan citra asli dengan citra blur untuk mendapatkan "Masker Tepi", lalu menambahkannya kembali ke citra asli.

01. Proses Masking

Formula:

$$M = I - \text{blur}(I, \sigma)$$

```
t0 = time.perf_counter()
blurred = manual_gaussian_filter(gray, sigma=sigma)
mask = gray - blurred
```

Keunggulan

Lebih fleksibel karena memiliki kontrol ganda (radius σ untuk ketebalan tepi dan faktor a untuk intensitas penajaman).

02. Sharpening Output

Formula:

$$I' = I + a \cdot M$$

di mana a adalah faktor penguatan tepi).

```
sharpened = np.clip(gray + a * mask, 0.0, 1.0)
elapsed = (time.perf_counter() - t0) * 1000
return dict(
    blurred = blurred,
    mask = normalize(mask),      # normalized for display
    sharpened = sharpened,
    elapsed = elapsed,
)
```


RUNTIME DAN BENCHMARKING

Menjalankan komputasi 9 metode secara paralel untuk membandingkan waktu eksekusi (Elapsed Time dalam milliseconds).

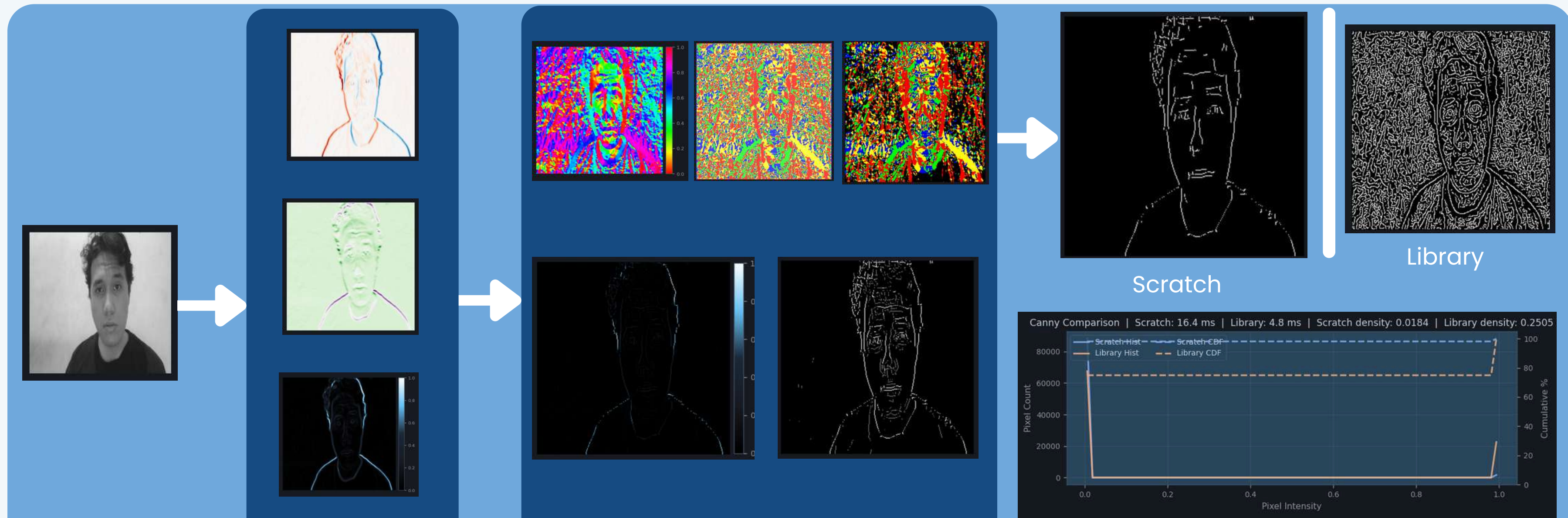


- Kuning/Hijau: Roberts, Sobel, dan Prewitt tercepat karena kernel kecil (2×2 dan 3×3).
- Merah/Jingga: Canny (Scratch), Kirsch, dan Extended Sobel memakan waktu komputasi lebih lama karena kompleksitas matriks dan operasi pelacakan (Hysteresis) berbasis CPU.

CONTOH #1

Canny Scratch vs Library dengan Input Gambar Sendiri

Implementasi manual secara visual menghasilkan tingkat akurasi piksel yang lebih baik daripada dengan library Skimage (tervalidasi dari Histogram/Ogive).

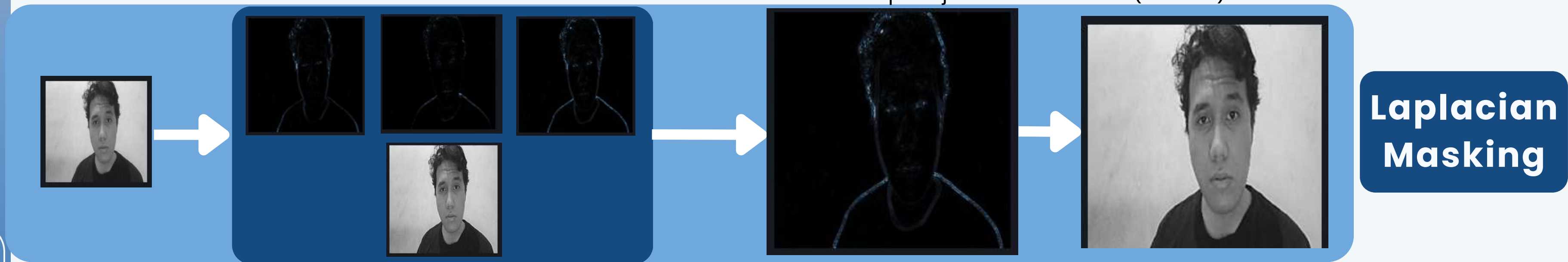


08

CONTOH #2

Image Sharpening (Laplacian dan USM) dengan Input Gambar Sendiri

Laplacian: Cenderung mengekstraksi detail mikro tapi rentan noise.
USM: Hasil penajaman lebih halus (natural) karena berbasis Gaussian blur.

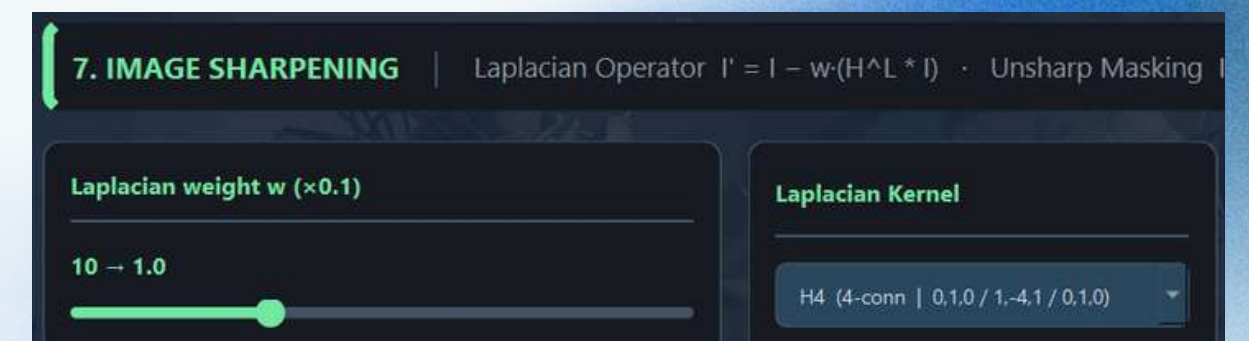
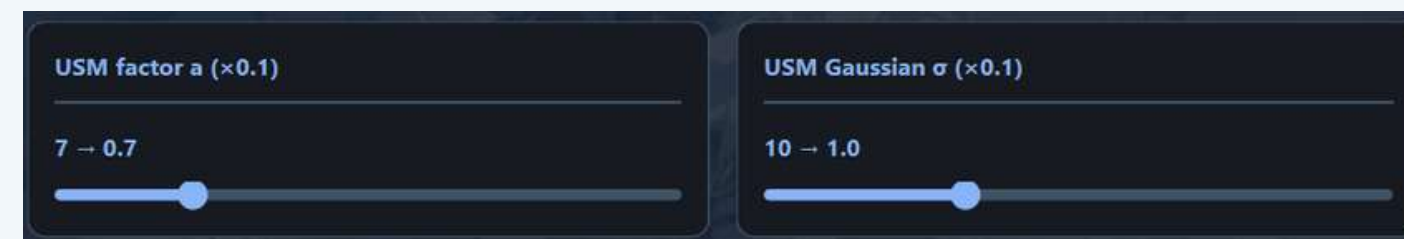
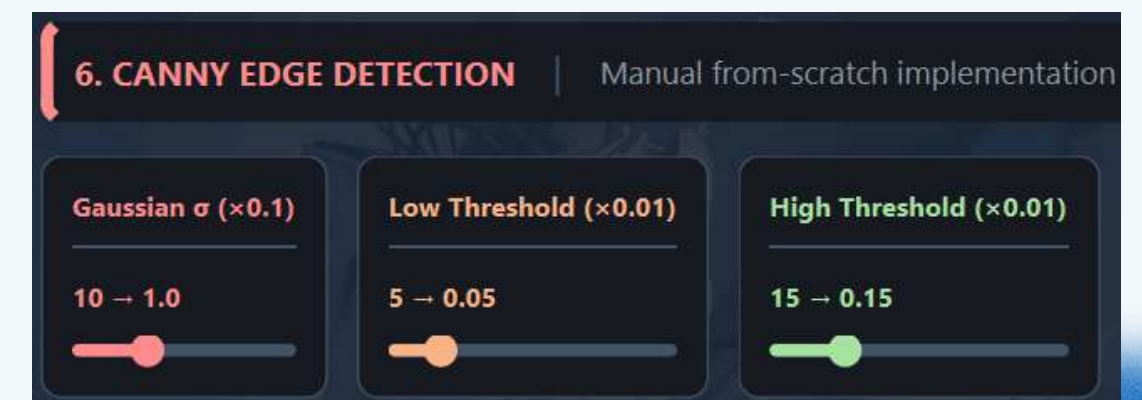


DAN CONTOH-CONTOH LAIN

[https://its.id/m/
Tugas2EdgeDetectionAndSharpeningJemi](https://its.id/m/Tugas2EdgeDetectionAndSharpeningJemi)

Catatan:

- Pada sub-tab Canny Edge Detection, bisa diubah-ubah parameter Gaussian σ , Low Threshold, dan High Thresholdnya.
- Pada sub-tab Image Sharpening, juga bisa diubah-ubah parameter Laplacian Weight w , Laplacian Kernel, USM Factor, dan USM Gaussian σ -nya.



Kesimpulan

Algoritma Canny berhasil dibangun dari nol menggunakan manipulasi vektorisasi matriks NumPy, membuktikan logika matematika tingkat lanjut dapat berjalan responsif di GUI.

Unsharp Masking (USM) memberikan kontrol penajaman/sharpening yang lebih adaptif pada citra medis/wajah dibandingkan Laplacian biasa karena parameter σ dan a dapat diatur terpisah.

Arsitektur Modular memfasilitasi komparasi runtime secara objektif, membuktikan trade-off antara kualitas deteksi tepi (seperti Canny) dengan kecepatan komputasi (seperti Roberts/Sobel).



THANK YOU

